



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO



SERLAB
Software Engineering Research

Ingegneria del Software

UML: Classi e Oggetti

ITPS A.A. 2025/2026

Vita Santa Barletta

Danilo Caivano

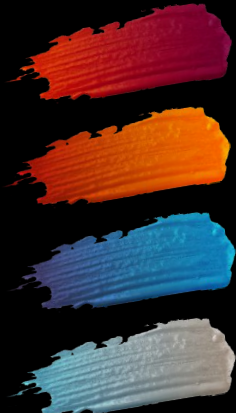
Antonio Piccinno

Dipartimento di Informatica - Università degli Studi di Bari

Via Orabona, 4 - 70125 - Bari

Tel: +39.080.5443270 | Fax: +39.080.5442536

serlab.di.uniba.it



Classe

- Descrittore di un insieme di oggetti che condividono gli stessi attributi, operazioni, relazioni e comportamento

The UML Reference Manual

- La **CLASSE** è un '**formato di oggetti**'
 - ❑ Definisce la struttura (caratteristiche) di tutti gli oggetti di quella classe
- Ogni oggetto è un'istanza di **esattamente** una sola classe

Classi di analisi

- Le classi di analisi sono classi che
 - ❑ Rappresentano un'astrazione ben definita nel dominio del problema
 - ❑ Devono corrispondere a concetti concreti del mondo del *business*

ContoBancario
numero proprietario saldo
deposito() prelievo()

Anatomia di una classe di analisi

- Un formato minimale per una classe di analisi comprende
 - ❑ **Nome**: obbligatorio
 - ❑ **Attributi**: i nomi degli attributi sono obbligatori
 - ❑ **Operazioni**: nell'analisi le operazioni possono anche essere solo delle affermazioni ad alto livello sulle responsabilità della classe. Si indicano i tipi restituiti e i parametri solo se necessari per rendere il modello più comprensibile
 - ❑ **Visibilità**: generalmente non la si esplicita
 - ❑ **Stereotipi**: si possono indicare se migliorano il modello
 - ❑ **Valori etichettati**: si possono indicare se migliorano il modello



Esempio di una classe di analisi

- **CarrelloSpesa** comprende, ad esempio, le seguenti responsabilità
 - ❑ Aggiungi un prodotto al carrello
 - ❑ Rimuovi un prodotto dal carrello
 - ❑ Mostra i prodotti nel carrello
- Ogni **responsabilità** ha lo stesso obiettivo: gestire il carrello della spesa del cliente
- **Responsabilità**: è un servizio che la classe offre ad altre classi



Qualità di una classe di analisi

- Il nome ne rispecchia l'intento
- È un'astrazione ben definita, che modella uno specifico elemento nel dominio del problema
- Ha un insieme ridotto e ben definito di responsabilità
- Ha la massima coesione interna
- Ha la minima interdipendenza con altre classi

Verifiche per una Classe di Analisi 1/2

- **Ogni classe deve avere tra tre e cinque responsabilità.** Per essere agevolmente gestita la coesione interna, per avere una buona manutenibilità e per evitare la proliferazione di classi
- **Nessuna classe può essere isolata.** Le classi devono cooperare tra loro per realizzare gli obiettivi del sistema a cui appartengono
- **Evitare di avere tante classi semplici.** La proliferazioni delle classi rende più difficile la comprensione del sistema che esse compongono
- **Evitare di avere poche classi troppo complesse.** La complessità delle classi degrada la comprensibilità di ogni classe e la manutenibilità della stessa



Verifiche per una Classe di Analisi 2/2

- **Evitare** di esprimere una classe come una funzione procedurale. Esse sono generate in genere da un approccio funzionale top-down. In genere, esse hanno una sola responsabilità; perciò disperdono l'espressione dei concetti, ovvero delle entità, che cooperano nel sistema
- **Evitare le classi onnipotenti.** Classi che sono al centro di tutto o che fanno tutto. Violano molti degli accorgimenti elencati prima. Conviene porre attenzione alle classi che hanno nel nome le parole come «**sistema**» e «**controllore**»
- **Evitare gli alberi di ereditarietà profondi.** Ogni strato dell'ereditarietà deve avere un preciso scopo; altrimenti, si sta creando decomposizione funzionale. In ogni caso un albero profondo di ereditarietà degrada la manutenibilità delle specifiche



Tecniche di miglioramento delle classi

- Quando un insieme di classi violano una o più punti di verifica, si possono utilizzare **tre tecniche per la riprogettazione**:
 - ❑ **Fusione**: mettere insieme responsabilità di più classi che siano coese tra loro
 - ❑ **Separazione**: separare responsabilità di una classe in più classi con l'accorgimento che ogni insieme di responsabilità estratto sia fortemente coeso al suo interno e lascamente coeso con gli altri insiemi estratti
 - ❑ **Combinazione lineare** delle prime due: la riprogettazione di alcune classi applicando una combinazione lineare di entrambe le prime due tecniche



Individuazione delle Classi

- Si raccolgono tutte le potenziali fonti di informazioni:
 - ❑ **Problemi**
 - ❑ **Bisogni**
 - ❑ **Modello dei casi d'uso**
 - ❑ **Glossario**
- Su ogni testo si fa un'analisi evidenziando:
 - ❑ **Nomi**: per esempio, **libro**
 - ❑ **Predicati nominali**: per esempio, **titolo del libro**
 - ❑ **Verbi**: per esempio, **prestare**
 - ❑ **Predicati verbali**: per esempio, **verificare la carta di credito**
- I **nomi** e i **predicati nominali** possono indicare **CLASSI** o **ATTRIBUTI DI CLASSI**
- I **verbi** e le **frasi verbali** possono indicare **RESPONSABILITÀ DELLE CLASSI**



Altre fonti di classi

- **Oggetti fisici** che tratta il sistema o interagiscono con esso; questi potrebbero essere tutte classi
- La **modulistica** utilizzata dall'organizzazione o dal dominio per cui si sta sviluppando il sistema potrebbero essere delle classi. Si deve tener conto che spesso i sistemi automatizzati servono per ridurre la modulistica
- **Interfacce** tra il sistema ed il mondo esterno, come: **schermi**, **tastiere**, **totem**, etc.
- **Entità concettuali** necessarie al funzionamento che non hanno una manifestazione materiale. Ad esempio: **IniziativaPromozione**



Oggetto

- Un'entità discreta, con confini ben definiti che incapsula stato e comportamento; un'istanza di una classe

The UML Reference Manual

- Un **OGGETTO** è un pacchetto coeso di dati e funzioni
 - ❑ L'unico modo di avere accesso alla parte di dati di un oggetto consiste nel chiamare una delle funzioni rese disponibili dall'oggetto stesso

Oggetto

Oggetto Banca



L'oggetto Banca invia il messaggio "preleva 150.00" a un oggetto Conto

Oggetto Conto



L'oggetto Conto risponde chiamando la sua operazione prelievo che decrementa il saldo di 150.00

messaggio
prelievo(150.00)



Oggetto

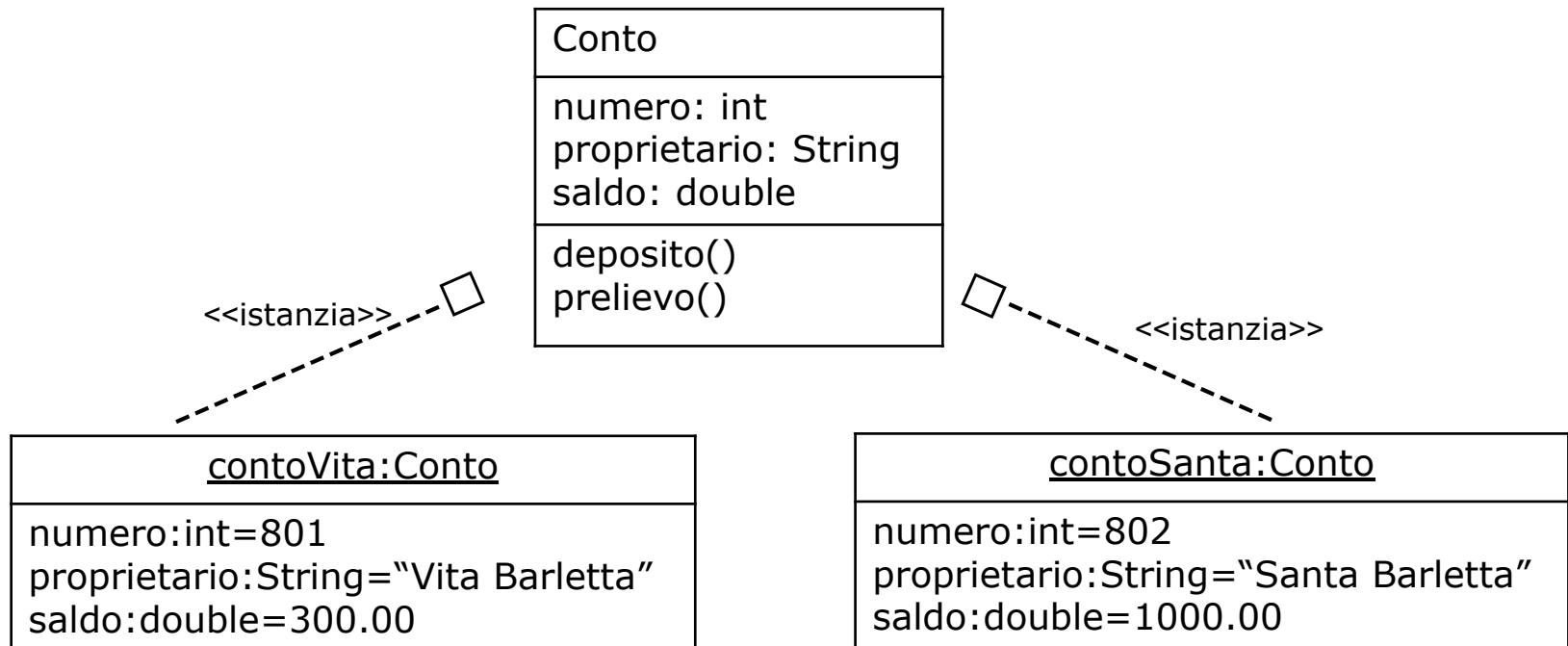
- Gli oggetti **collaborano** per eseguire le funzioni del sistema, stabilendo dei **collegamenti** tra loro e scambiandosi messaggi lungo questi collegamenti
- Quando un oggetto riceve un messaggio, esso esamina le sue operazioni (o metodi, se si parla di progettazione) per trovarne una la cui **segnatura** corrisponda alla segnatura del messaggio. Se tale operazione esiste, allora l'oggetto la esegue
- Le segnature sono composte dal **nome del messaggio** (o dall'operazione), **dai tipi dei parametri** e dal **valore restituito**
- La struttura run-time di un sistema OO è costituita da numerosi oggetti che vengono creati, esistono per un certo tempo e poi, vengono distrutti



Classi e oggetti

- **Relazione:** “Una connessione tra elementi del modello”

The UML Reference Manual

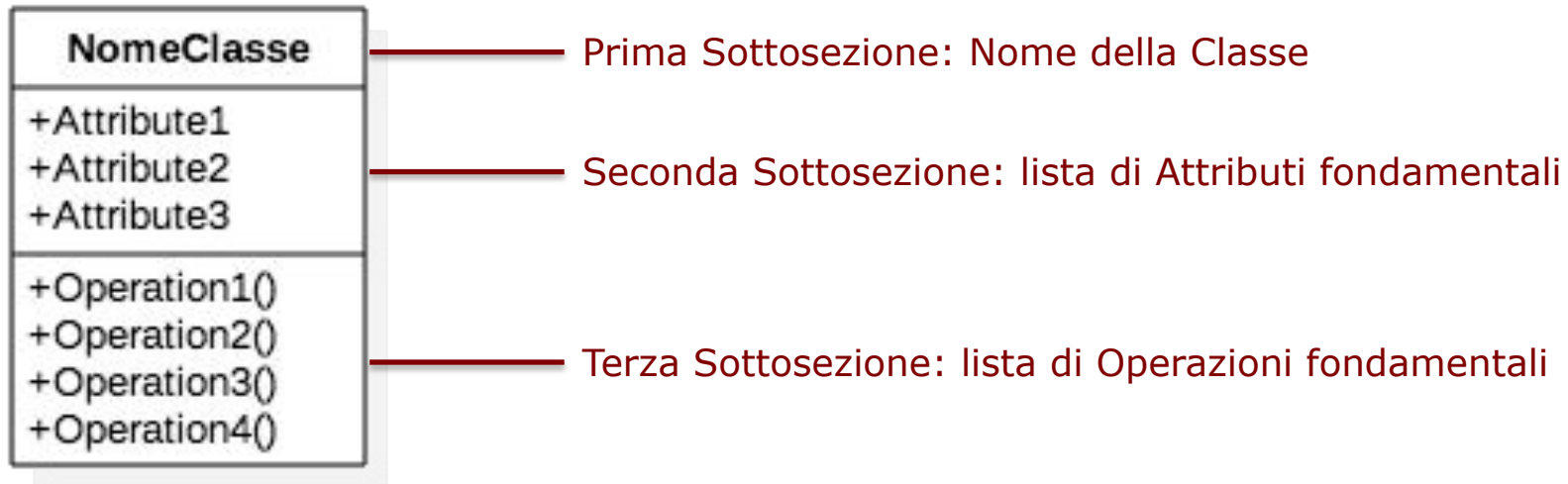


Classi e oggetti

- Una classe definisce la struttura (l'insieme delle caratteristiche) di tutti gli oggetti di quella classe
- Tutti gli oggetti della stessa classe devono avere **lo stesso** insieme di **operazioni**, **lo stesso** insieme di **attributi** e **lo stesso** insieme di **relazioni**, ma possono avere **diversi valori** degli **attributi**



Classe: Notazione



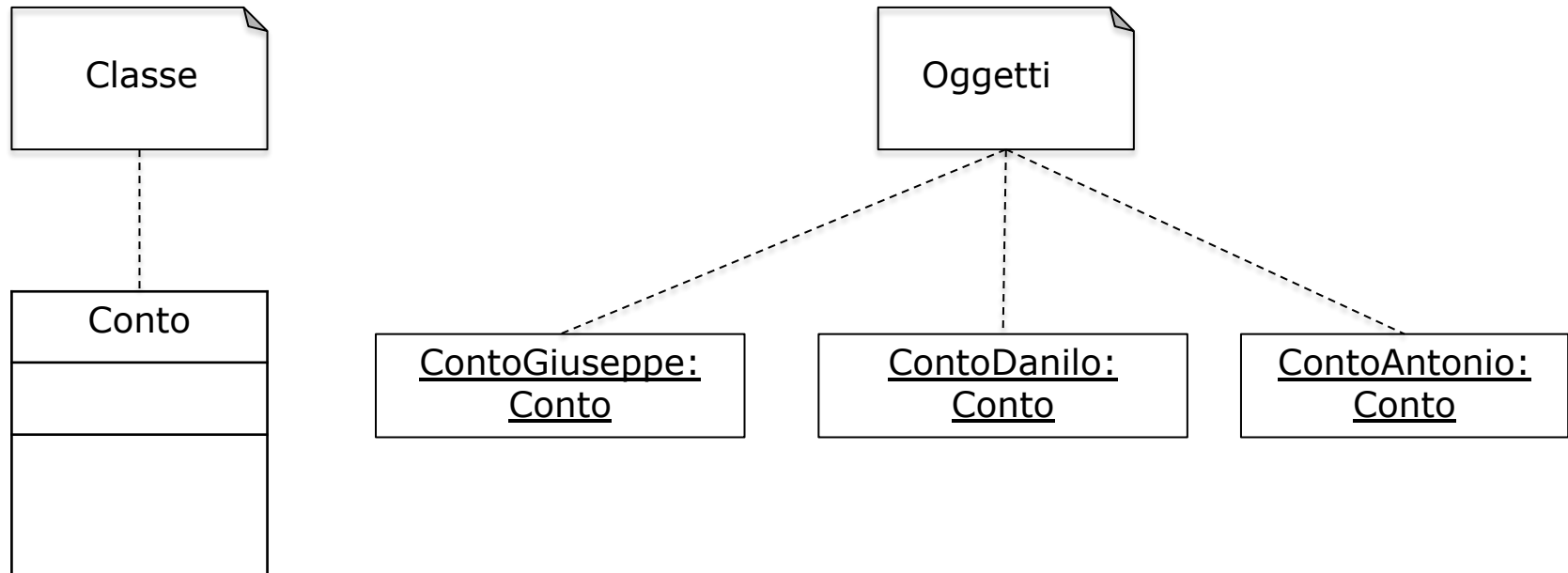
Oggetto: Notazione

NomeOggetto:NomeClasse

Un oggetto è rappresentato con lo stesso simbolo usato per rappresentare le classi con la precisazione che il nome della classe è sottolineato ed è preceduto dal simbolo ":" per indicare che si tratta di un oggetto



Classi e Oggetti: esempio

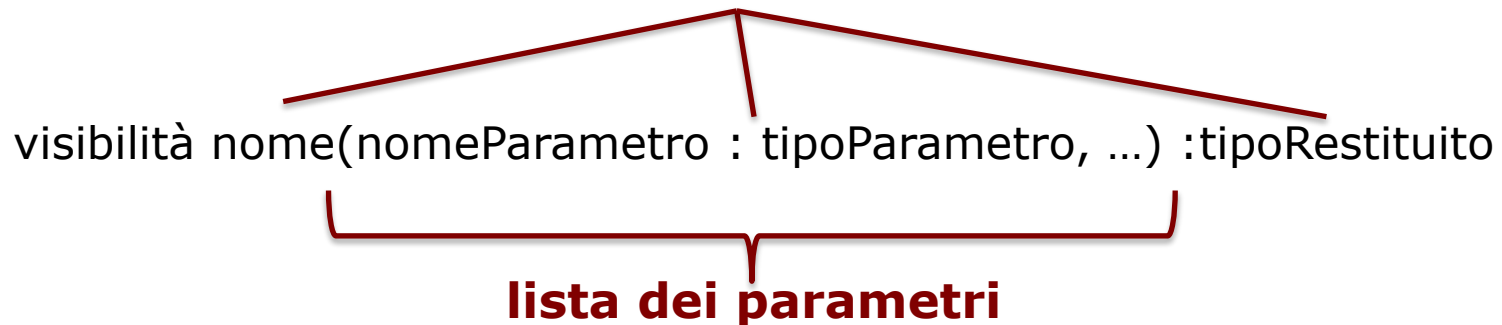


- Un diagramma degli oggetti è un'istanza del diagramma delle Classi
- Mostra un'istantanea del diagramma delle Classi in un dato istante di tempo

Classe: esempio

Tela
+Attributo1
+disegnaCerchio(origine:Punto=Punto(00),raggio:Inter) +disegnaQuadrato(origine:Punto=Punto(00),area:Dimensione)

Segnatura dell'operazione



- La **segnatura** di una operazione è costituita da:
 - ❑ il nome del metodo
 - ❑ il numero e i tipi (**non i nomi**) dei parametri formali

Visibilità degli elementi di una classe

- Attributi ed operazioni di una classe sono caratterizzati da uno dei seguenti livelli di accesso (visibilità) rispetto alle altre classi

Simbolo UML (Ornamento)	Classificatore	Semantica
+	Public	L'elemento è visibile a qualunque classe: ogni elemento che può accedere alla classe, può anche accedere a ogni suo membro con visibilità pubblica
-	Private	L'elemento non è visibile ad alcuna classe: solo le operazioni della classe possono accedere ai membri con visibilità privata
#	Protected	L'elemento è visibile a tutte le sotto-classi della propria classe di definizione
~	Package	La classe è visibile a tutte le classi del package di appartenenza mentre è privata per tutte le classi esterne a tale package (definita in JAVA)

Implementazione fisica di una classe

- Esiste una **corrispondenza tra** la **rappresentazione** UML di una classe e la sua **implementazione** in un linguaggio Object Oriented

Persona
-nome: string -cognome: string -eta: int
+getNome() : string +setNome(nome) : void ...



```
public class Persona {  
    private String nome;  
    private String cognome;  
    private int eta;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    ...  
}
```

DIAGRAMMA delle CLASSI

Diagramma delle classi

- Grafico che fornisce una vista strutturale del sistema in termini di
 - ❑ Classi
 - ❑ Attributi
 - ❑ Operazioni
 - ❑ Relazioni tra classi
- **Osservazione**
 - ❑ **Non fornisce** alcuna informazione temporale

Relazioni tra classi

- Le principali relazioni tra classi sono:
 - ☐ Associazione
 - ☐ Aggregazione
 - ☐ Composizione
 - ☐ Classe associativa
 - ☐ Generalizzazione
 - ☐ Dipendenza

Relazione di ASSOCIAZIONE

- **Definizione**

- ☐ Indica la presenza di una popolazione di legami relazionali tra istanze di classi

- **Proprietà**

- ☐ È sempre bi-direzionale
- ☐ Può essere binaria (coinvolge due classi) o n-aria (coinvolge tre o più classi)

- **Osservazione**

- ☐ Un'associazione binaria è un caso speciale di relazione n-aria con una propria notazione

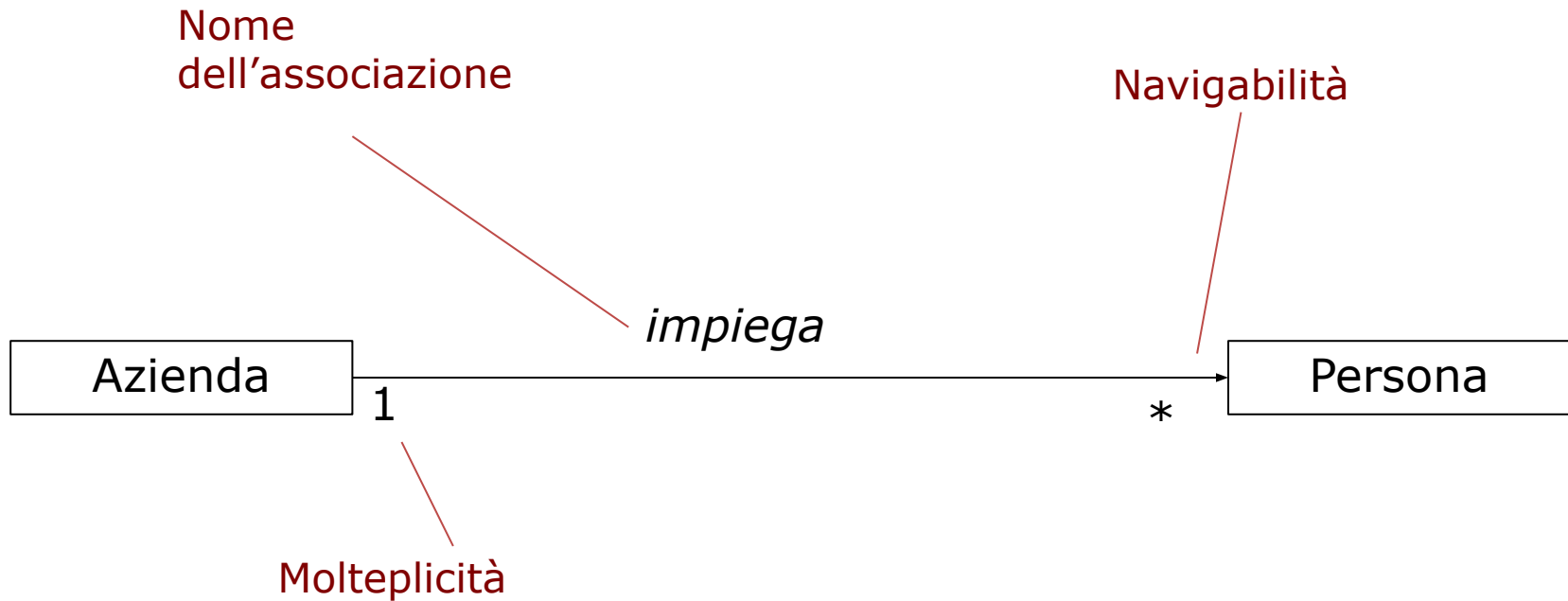


Sintassi per la relazione di ASSOCIAZIONE

- Le associazioni possono essere indicate con:
 - ❑ **Un nome dell'associazione**
 - Verbo o frase verbale poiché indica un'azione che l'oggetto origine esegue sull'oggetto destinazione
 - ❑ **Nomi dei ruoli**
 - Nome o frase nominale in quanto descrivono il ruolo che gli oggetti possono ricoprire
 - ❑ **Molteplicità**
 - Numero di oggetti di una classe che possono partecipare in una relazione in un dato istante
 - ❑ **Navigabilità**
 - Indica che è possibile spostarsi da un qualsiasi oggetto della classe origine a uno o più oggetti della classe destinazione, a seconda della molteplicità



Relazione di ASSOCIAZIONE



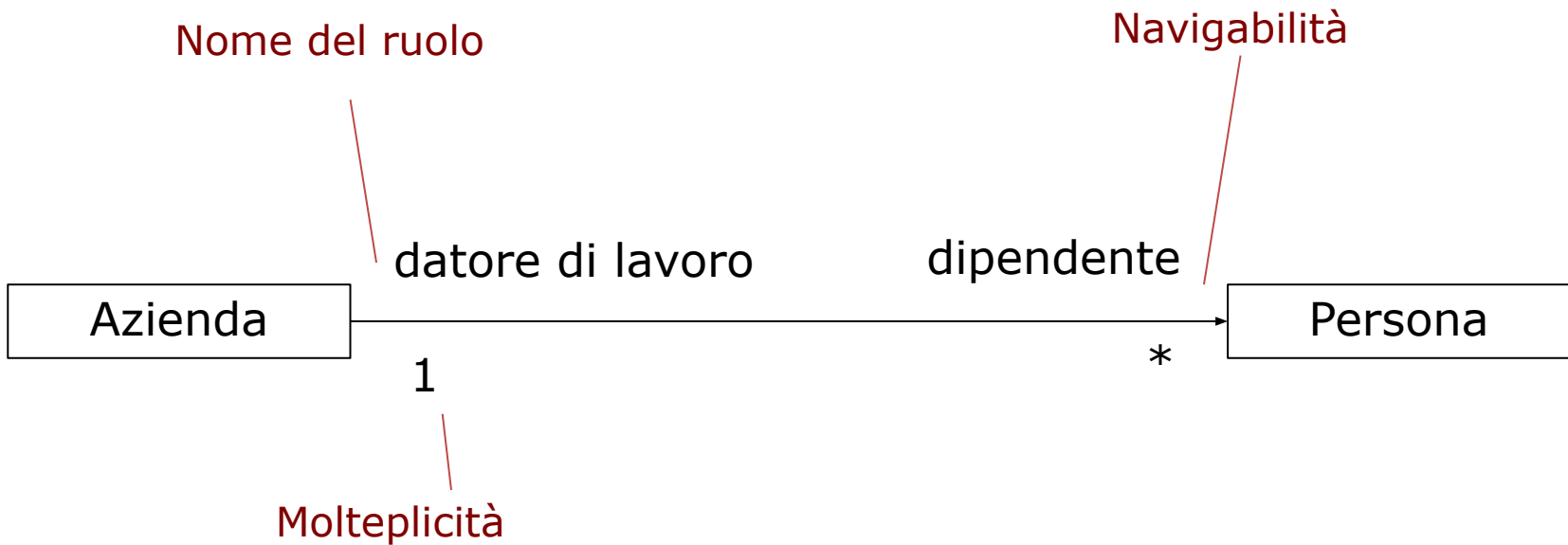
«un'**Azienda** impiega molte **Persone**»

«ogni **Persona** è impiegata da una sola **Azienda**»
in un dato istante

Ruoli nella relazione di ASSOCIAZIONE

- I ruoli forniscono una **modalità per attraversare le relazioni** da una classe ad un'altra
- **Generalità**
 - ❑ Possono essere usati in alternativa ai nomi delle associazioni
 - ❑ Sono spesso usati per relazioni tra oggetti della stessa classe

Ruoli nella relazione di ASSOCIAZIONE

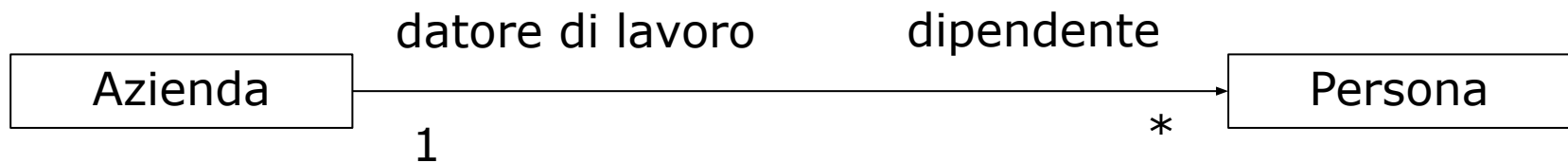


«**Azienda** ha il ruolo di **datore di lavoro**»

«**Persona** ha il ruolo di **dipendente**»

Molteplicità nella relazione di ASSOCIAZIONE

- Indica quante istanze di una classe possono essere associate con una singola istanza di un'altra classe
- **Osservazione**
 - ❑ Se la molteplicità non è indicata non si opera alcuna assunzione



Molteplicità nella relazione di ASSOCIAZIONE

- In un dato istante un oggetto **Persona** è dipendente di esattamente un oggetto **Azienda**



- Tale vincolo esprime due regole di business
 1. Gli oggetti **Persona** possono essere dipendenti di un'**Azienda** per volta
 2. Gli oggetti **Persona** devono sempre essere dei dipendenti

Molteplicità nella relazione di ASSOCIAZIONE

- **Notazione**

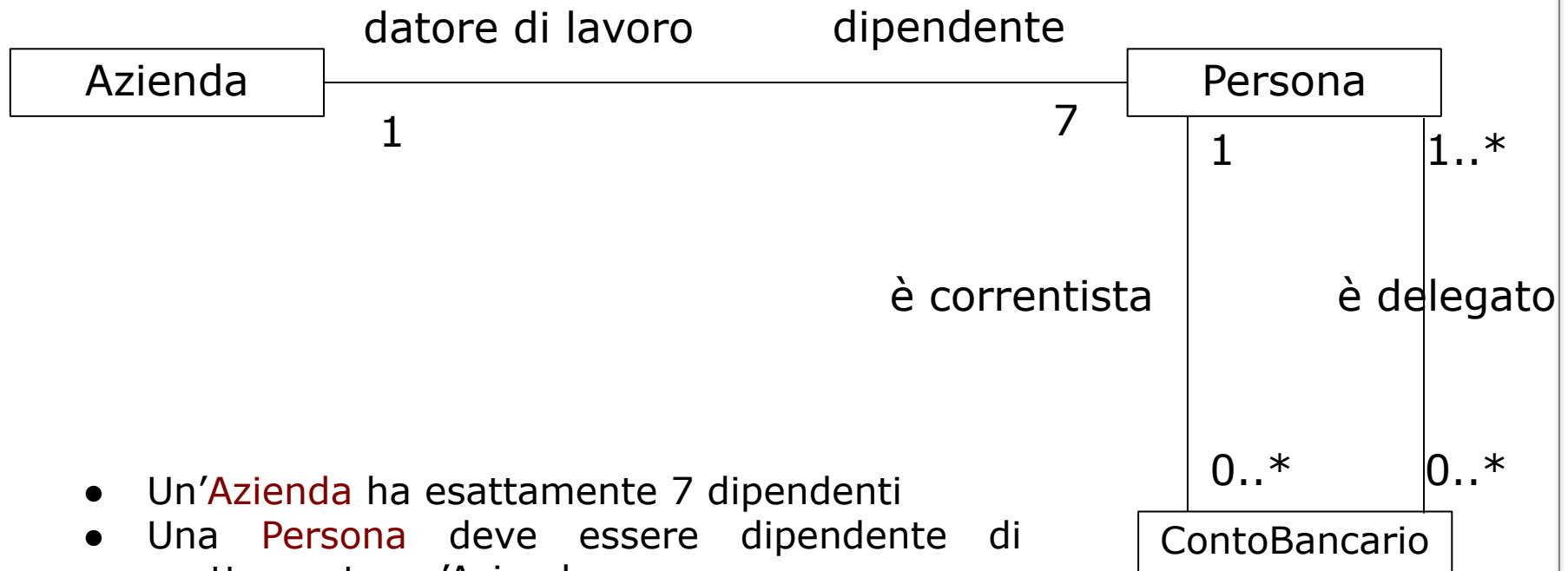
- ❑ Stringa di testo che comprende sequenze di intervalli di interi separate da una virgola
- ❑ Un intervallo, potenzialmente illimitato, è nel formato
 - **limite inferiore..limite superiore**

- **Esempi di molteplicità**

- ❑ 0..1 (da zero ad una istanza)
- ❑ 0..* (zero o più istanze)
- ❑ 1..* (una o più istanze)
- ❑ 1..6 (da una a sei istanze)
- ❑ 1..3, 7..10, 15, 19..*
- ❑ 1 (esattamente uno)
- ❑ * (zero o più)

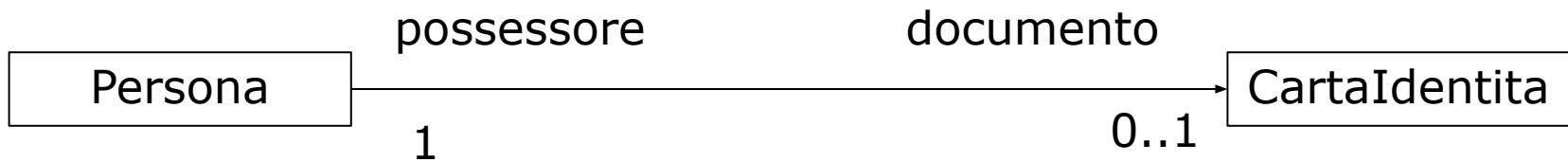
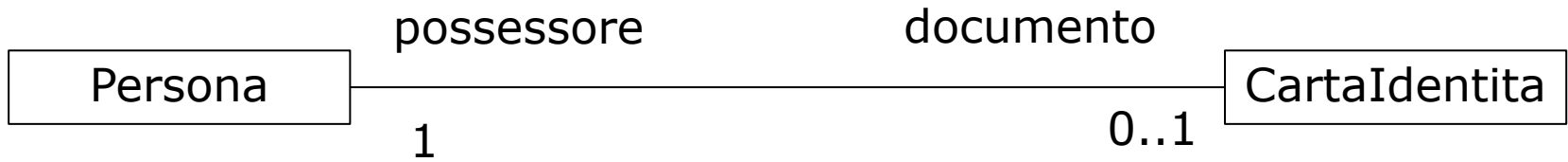


Esempio: Molteplicità nella relazione di Associazione



- Un'Azienda ha esattamente 7 dipendenti
- Una Persona deve essere dipendente di esattamente un'Azienda
- Un ContoBancario può avere un solo correntista
- Un ContoBancario può avere uno o più delegati
- Una Persona può essere correntista di zero o più di un ContoBancario
- Una Persona può essere delegato di zero o più di un ContoBancario

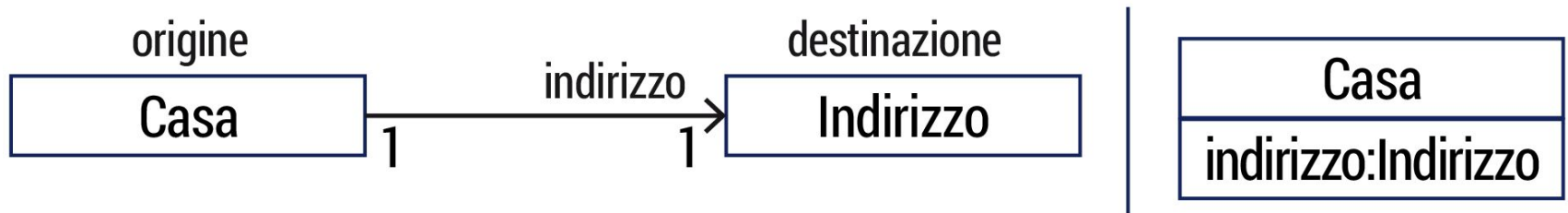
Navigabilità nella relazione di ASSOCIAZIONE



Associazioni e attributi

- Un'associazione tra una classe origine e una destinazione indica che gli oggetti della prima possono avere riferimenti agli oggetti della seconda
- Un altro modo di vedere l'associazione è che la classe sorgente possiede un **pseudo-attributo** il cui tipo è la classe destinazione

Associazioni e attributi



Se una relazione navigabile ha un ruolo, allora è come se la classe origine avesse uno pseudo-attributo con lo stesso nome del ruolo e con lo stesso tipo della classe destinazione.

Associazioni e attributi



```
public class Casa {  
    private Indirizzo indirizzo;  
}
```

- Una classe **Casa** contenente un attributo **indirizzo**, di tipo **Indirizzo**
- Il nome del ruolo diviene il nome dell'attributo e la classe destinazione dell'associazione diventa la classe dell'attributo

Associazioni e attributi

- Le molteplicità superiori ad 1 si realizzano come:
 - ❑ Un **attributo di tipo array** oppure
 - ❑ Un **attributo** che abbia come **tipo** una qualche **classe collezione**

Implementazione Associazione

- **Generalità**

- ❑ **Non esiste un *mapping diretto*** su un costrutto di un linguaggio di programmazione
- ❑ Può essere implementata introducendo degli attributi *ad hoc* che contengono dei **riferimenti** alle istanze delle classi associate
- ❑ Il riferimento a più istanze di una classe può essere modellato con liste, array, ...
- ❑ In fase di implementazione si può decidere di codificarla solo in una delle due direzioni

Implementazione Associazione

- In ogni classe partecipante all'associazione si definisce un'operazione **"associa"** la cui semantica è "crea un riferimento all'altra classe"
- L'associazione, di fatto, si realizza a **run-time** all'interno di un'altra classe che
 - ❑ passa, tramite l'operazione *associa*, ad ogni oggetto, partecipante all'associazione, i riferimenti degli altri oggetti

Implementazione 1 a 1 (*static-time*)

```
class A {  
    private B associazioneX; //nome dell'associazione o del ruolo  
    public associaPartner(B partner);  
}
```

```
class B {  
    private A associazioneX; //nome dell'associazione o del ruolo  
    public associaPartner(A partner);  
}
```

Relazione 1 a 1 bidirezionale: implementazione in JAVA (static-time)

```
public class CartaIdentita {
    // altri attributi
    private Persona possessore; // nome ruolo
    private String numero;
    //altri metodi
    public void associaPersona(Persona partner) {
        //todo
    };
    public CartaIdentita (String numcarta) {
        //todo
    };
}

public class Persona {
    private CartaIdentita documento; // nome ruolo
    private String nome;
    // ... altri attributi ed operazioni
    public void associaDocumento(CartaIdentita partner) {
        //todo
    };
}
```

Relazione 1 a 1 bidirezionale: implementazione in JAVA (static-time)

```
// classe GestioneIdentita realizza l'associazione bidirezionale nei metodi
getCartaIdentita e getPersona
public class GestioneIdentita {
    // attributi di gestionePersona
    // operazioni di gestionePersona
    public CartaIdentita getCartaIdentita() {
        CartaIdentita documento = new CartaIdentita("12456");
        Persona pers = new Persona();
        documento.associaPersona(pers);
        return documento;
    }
    public Persona getPersona() {
        Persona persona = new Persona();
        CartaIdentita documento = new CartaIdentita("12456");
        persona.associaDocumento(documento);
        return persona;
    }
}
```



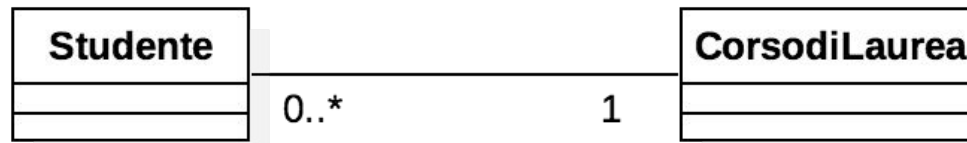
Associazione 1 a molti

- Un'associazione uno a molti tra una classe A ed una classe B descrive una connessione semantica per cui a ciascun oggetto di A possono corrispondere zero, uno o più oggetti di B mentre ad un oggetto di B corrisponde al più un oggetto di A
 - ❑ La classe che partecipa con molteplicità 1 nell'associazione rappresenta la relazione attraverso una variabile di tipo lista di riferimenti alla classe associata

Implementazione 1 a molti (*static-time*)

```
class A {  
    private B Contenitoreb; // contenitore di oggetti di tipo B  
    public   aggOggetto (B newobj);  
}  
  
class B {  
    private A associazioneX; //nome dell'associazione o del ruolo  
    public   associaPartner(A partner);  
}
```

Relazione 1 a molti: implementazione in JAVA (*static-time*) 1/2



```
import java.util.ArrayList;
```

```
/* esempio in codice JAVA di implementazione associazione 1 a molti con navigabilità da CorsodiLaurea a
Studente */
```

```
public class CorsodiLaurea {
    private List<Studente> studenti;
    private String denominazione;
    // altri attributi

    public CorsodiLaurea (String denomCorso) {
        this.denominazione = denomCorso;
        this.studenti = new ArrayList();
    };

    public void addStudente (Studente newstud){
        studenti.add(newstud);
    };

    public void addAllStudenti (ArrayList<Studente> studenti){
        this.studenti = studenti; // In alternativa potete usare this.studenti.addAll(studenti)
    };
}
```



Relazione 1 a molti: implementazione in JAVA (*static-time*) 2/2

```
import java.util.ArrayList;
import java.util.List;

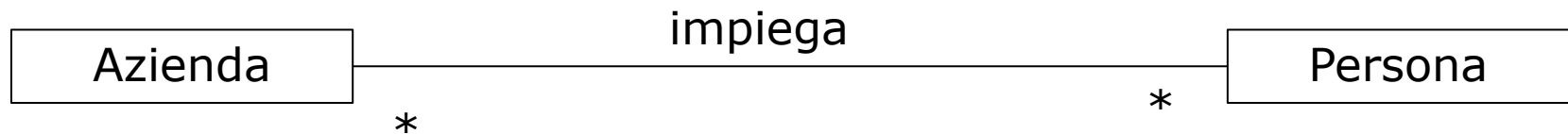
public class GestioneLaurea {
    // attributi e metodi della classe GestioneLaurea
    public CorsodiLaurea getCdL(String nome) {
        CorsodiLaurea cd1 = new CorsodiLaurea(nome);
        List<Studiante> studenti = new ArrayList<Studiante>();
        Studiante st1= new Studiante("pluto","12345");
        Studiante st2= new Studiante("paperino","12346");
        Studiante st3= new Studiante("pippo","12347");
        studenti.add(st1);
        studenti.add(st2);
        studenti.add(st3);
        cd1.allStudenti(studenti);
        return cd1;
    };
}
```

Classe Associativa



- Ogni oggetto **Persona** può essere dipendente di molti oggetti **Azienda**
- Ogni oggetto **Azienda** può impiegare molti oggetti **Persona**
- Non si può modellare lo stipendio di una persona come attributo della classe **Persona**, perché ogni istanza di **Persona** potrebbe lavorare per diverse **Aziende** percependo uno stipendio diverso da ciascuno di esse

Classe Associativa



- Stipendio
 - ❑ È ***una proprietà dell'associazione stessa***
 - ❑ Per ogni associazione **impiega** che un oggetto Persona ha con un oggetto Azienda, c'è uno stipendio per uno specifico rapporto di lavoro

CLASSE ASSOCIATIVA

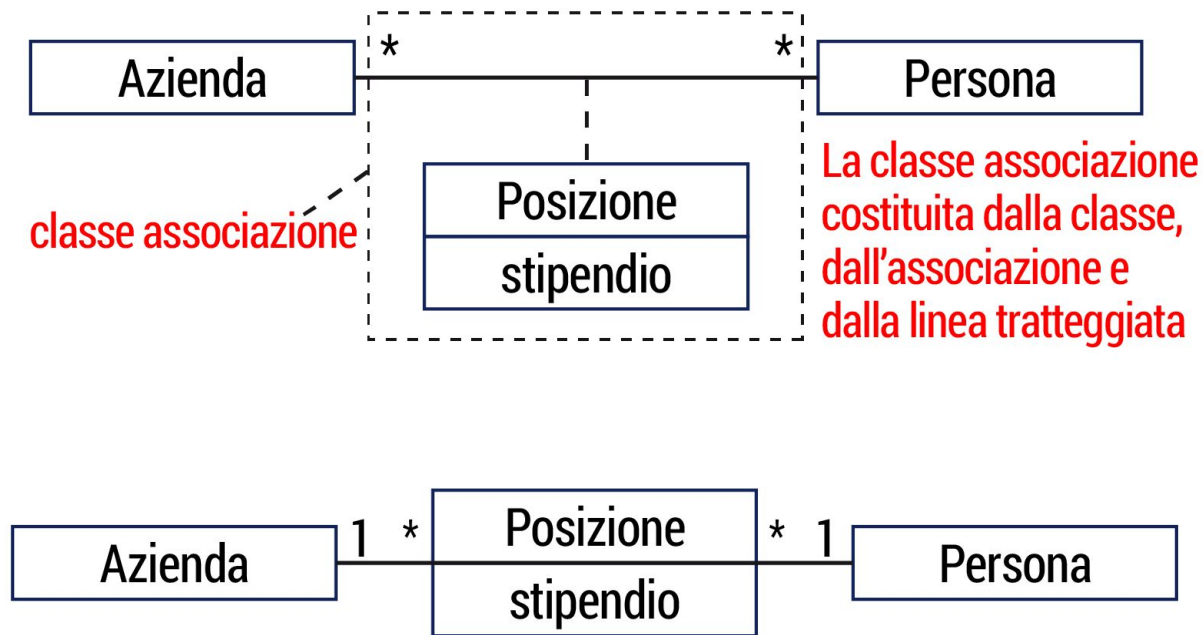


Classe Associativa

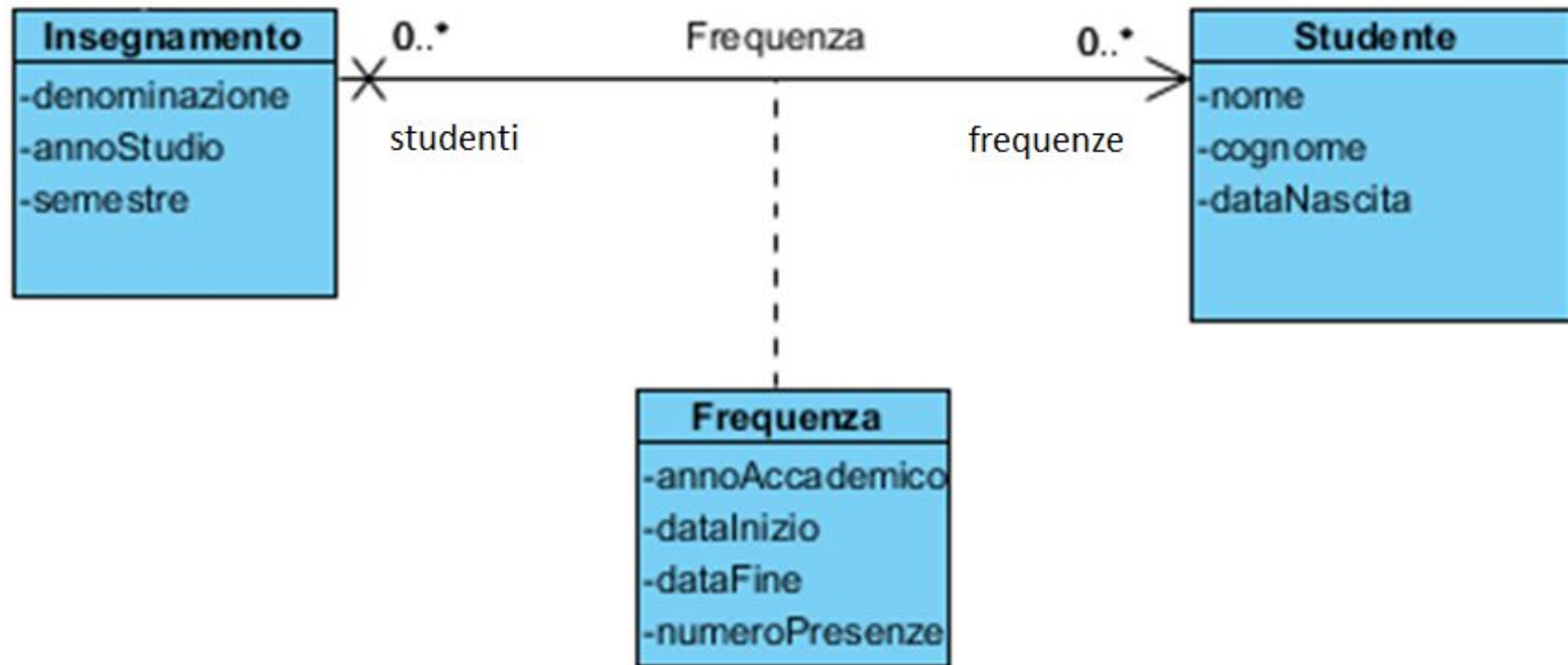
- Le proprietà di un'associazione sono proprie della relazione e non delle classi coinvolte, pertanto può essere opportuno definire una classe associativa
 - ❑ Utilizzata, in genere, nelle associazioni "uno a molti" e "molti a molti"
- **Proprietà**
 - ❑ La classe associativa contiene un riferimento ad ognuno degli oggetti in relazione (*doppia navigabilità*)
 - ❑ Ogni oggetto in relazione contiene un riferimento ad un oggetto della classe associativa (*doppia navigabilità*)



Classe Associativa



Esempio: Classe Associativa



Classe Associativa: implementazione in JAVA 1/3

```
public class Insegnamento {  
    private int semestre, annoStudio;  
    private String denominazione;  
    private List<Frequenza> studenti;  
  
    public Insegnamento(String codice) {  
        this.codice = codice;  
        this.studenti = new ArrayList<Frequenza>();  
    }  
  
    public addAllFrequenze(ArrayList<Frequenza> frequenze) {  
        this.frequenze = frequenze;  
    }  
  
    public addFrequenza(Frequenza frequenza) {  
        this.studenti.Add(frequenza);  
    }  
}
```



Classe Associativa: implementazione in JAVA 2/3

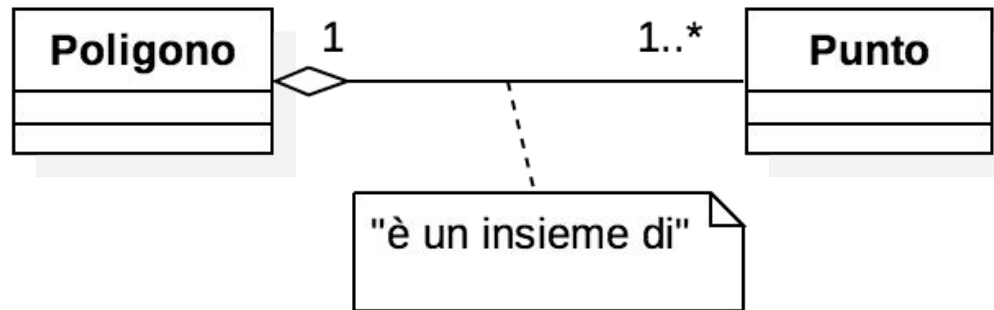
```
public class Studente {  
    private String nome, cognome;  
    private Date dataNascita;  
    private List<Frequenza> insegnamenti;  
  
    public Studente() {  
        this.insegnamenti = new ArrayList<Frequenza>();  
    }  
  
    public allAllInsegnamenti(ArrayList<Frequenza> insegnamenti) {  
        this.insegnamenti = insegnamenti;  
    }  
  
    public addInsegnamento(Frequenza insegnamento) {  
        this.insegnamenti.Add(insegnamento);  
    }  
}
```



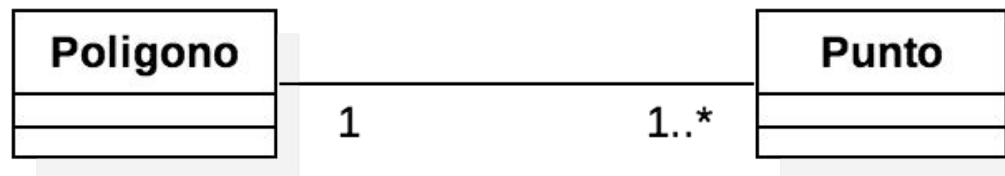
Classe Associativa: implementazione in JAVA 3/3

```
public class Frequenza {  
    public String annoAccademico;  
    public Date dataInizio, dataFine;  
    public int numeroPresenze;  
    private Studente studente;  
  
    private Insegnamento insegnamento;  
  
    public Frequenza(Studente student, Insegnamento insegnamento) {  
        this.studente = student;  
        this.frequenza = insegnamento;  
    }  
}
```


Relazione di AGGREGAZIONE



⇒ Tale rappresentazione equivale a



Relazione di AGGREGAZIONE

- Relazione di associazione che definisce una relazione tra un “tutto” ed un insieme di parti di cui il “tutto” è costituito (*whole-part relationship*)
- L'aggregazione è la relazione “**parte-di**”
 - ❑ Esempio: una macchina ha un motore e quattro ruote come sue parti
 - ❑ Un oggetto (il **TUTTO**) utilizza i servizi di un altro oggetto (**LA PARTE**)
 - ❑ Il **TUTTO** tende ad essere l'elemento che **domina** e controlla la relazione
 - ❑ La **PARTE** di solito si limita a **servire** le richieste del TUTTO ed è quindi più passiva



Semantica dell'AGGREGAZIONE

- L'aggregato può in alcuni casi esistere indipendentemente dalle parti, ma in altri casi no
- Le parti possono esistere indipendentemente dall'aggregato
- L'aggregato è in qualche modo incompleto se mancano alcune delle sue parti

Relazione di AGGREGAZIONE

- **Proprietà**

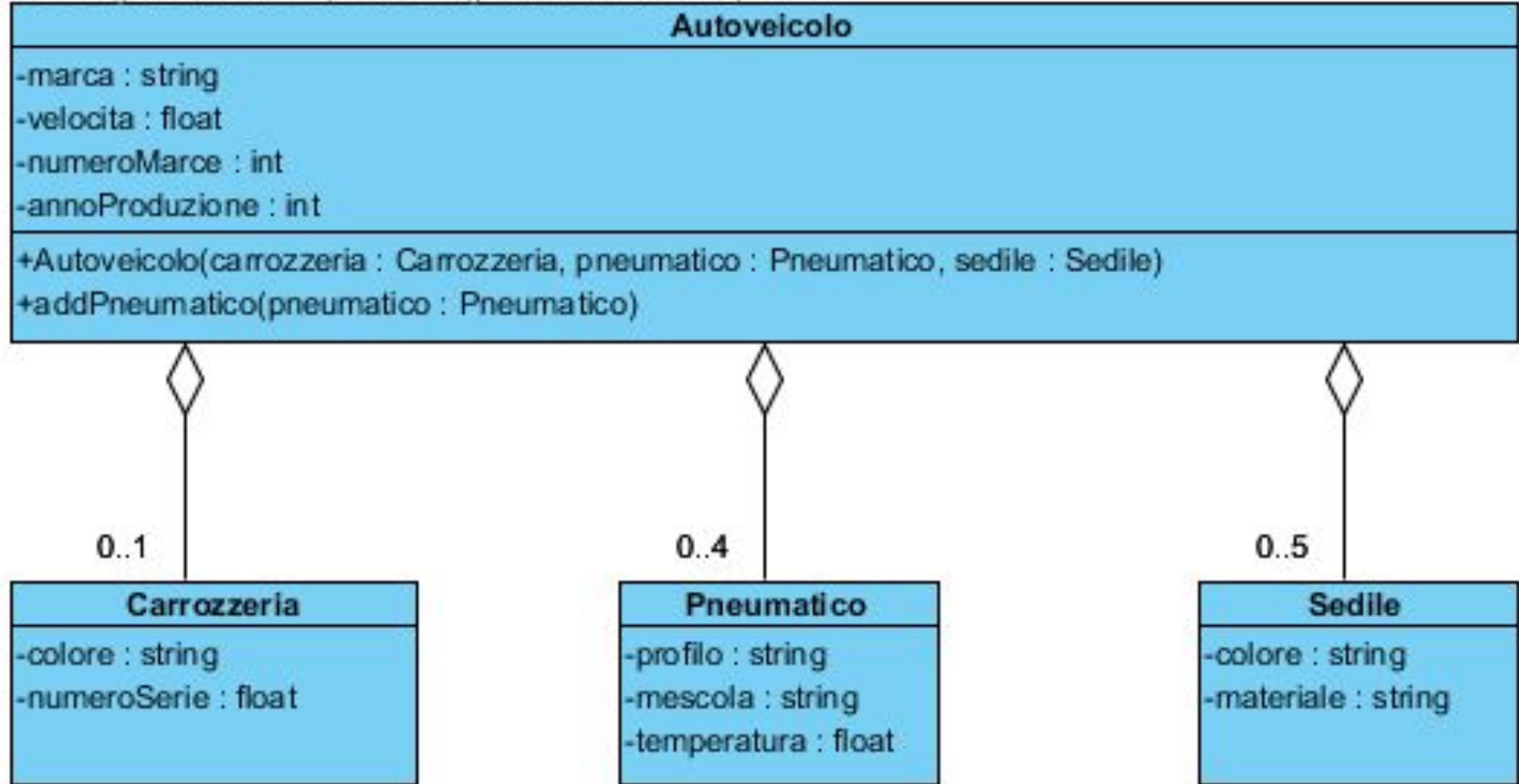
- ☐ gode delle proprietà transitiva e antisimmetrica
- ☐ a *run-time* un oggetto contenuto **sopravvive** all'oggetto contenitore

- **Esempi**

- ☐ Autoveicolo composto da Carrozzeria, Pneumatico, ...
- ☐ Impresa composta da Aree, Unità, ...
- ☐ PC composto da Mouse, Tastiera, Monitor, Piastra madre
- ☐ Piastra madre composta da CPU, hard disk, floppy disk, ...

Esempio: Relazione di Aggregazione

Visual Paradigm Standard Edition (Università degli Studi di Bari Aldo Moro)



Aggregazione: implementazione

- Per realizzare l'aggregazione è necessario
 - ❑ Definire degli attributi (privati) del tipo delle classi aggregate o, in alternativa, definire una opportuna struttura attraverso la quale mantenere le istanze aggregate
 - La dimensione di ogni attributo dipende dalla molteplicità
 - ❑ Ottenere i riferimenti alle istanze aggregate e mapparle con i corrispondenti attributi privati
 - Costruttori, metodi *add*
 - ❑ Definire, **se previsto dalle specifiche**, delle operazioni per esportare i riferimenti delle istanze aggregate

Aggregazione: implementazione in JAVA 1/3

```
package domain;
import java.util.ArrayList;
import java.util.Iterator;
public class Autoveicolo {
    private String marca;
    private float velocita;
    private int numeroMarce;
    private int annoProduzione;
    private Carrozzeria carrozzeria; // riferimento oggetto aggregato
    private List<Pneumatico> pneumatici; // riferimento oggetto aggregato
    private List<Sedile> sedili; // riferimento oggetto aggregato
```

Aggregazione: implementazione in JAVA 2/3

```
// il costruttore riceve dei riferimenti per ognuno degli oggetti in relazione
public Autoveicolo(Carrozzeria carrozzeria, ArrayList<Pneumatico> pneumatici,
ArrayList<Sedile> sedile) {
    // setting dei riferimenti degli oggetti
    this.carrozzeria = carrozzeria;
    this.pneumatici = pneumatici;
    this.sedili = sedili;
}

// riceve un riferimento ad un oggetto di tipo pneumatico
public void addPneumatico(Pneumatico pneumatico) {
    pneumatici.add(pneumatico);
}
}
```


Aggregazione: implementazione in JAVA 3/3

```
package domain;
import java.util.ArrayList;
public class GestioneAuto {
    private Autoveicolo auto;
    private Carrozzeria carrozzeria;
    private ArrayList<Pneumatico> pneumatici;
    private ArrayList<Sedile> sedili;
    //altri attributi
    // il metodo che realizza l'aggregazione può essere uno qualunque; ce ne
    potrebbe essere più di uno;
    public void gestioneAuto() {
        carrozzeria = new Carrozzeria();
        // pneumatici e sedili andrebbero opportunamente istanziati nei
        costruttori
        pneumatici = new ArrayList<Pneumatico>();
        sedili = new ArrayList<Sedile>();
        auto = new Autoveicolo( carrozzeria, pneumatici, sedili);
    }
    // altre operazioni
}
```

Relazione di COMPOSIZIONE

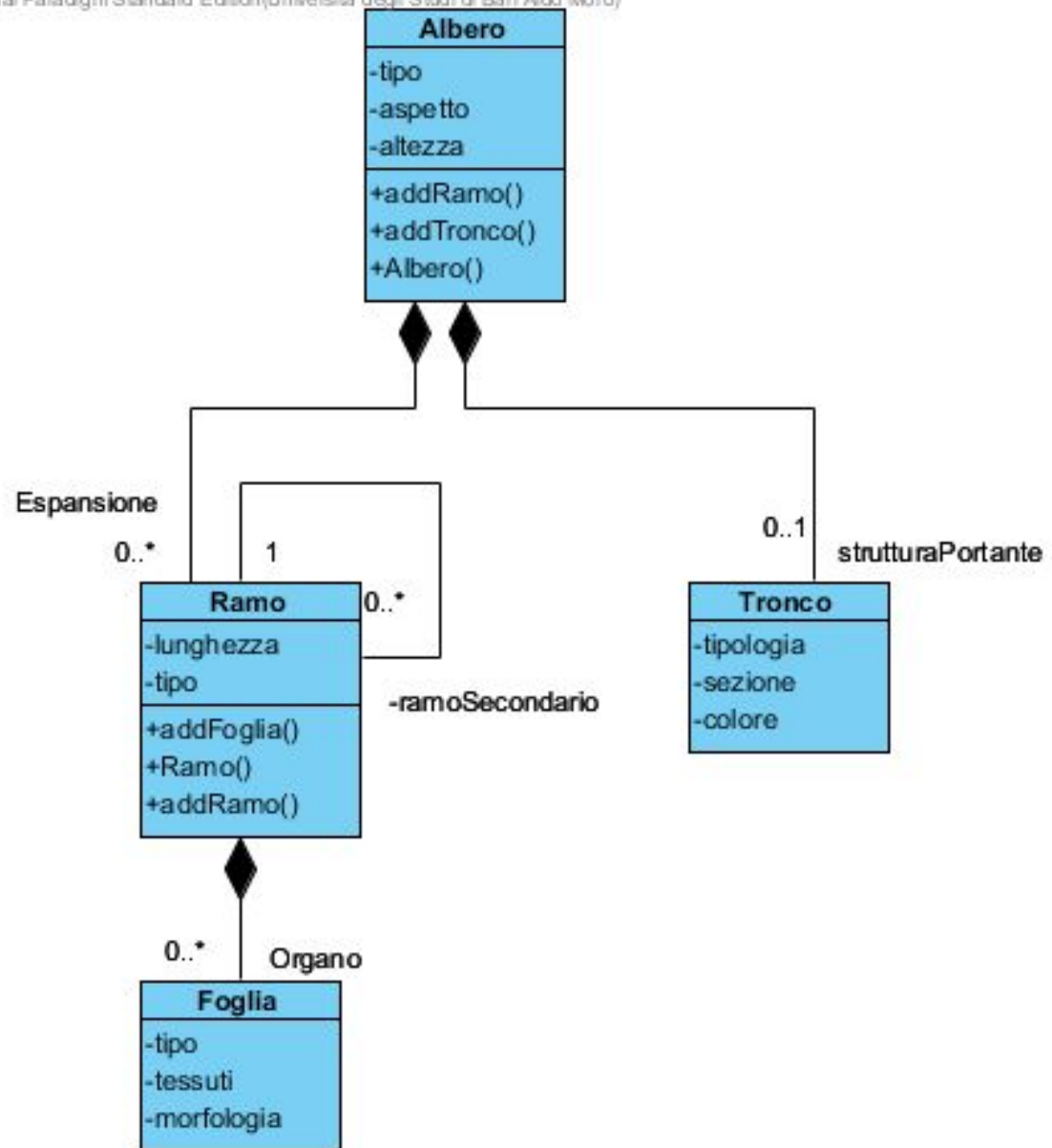
- **Relazione di aggregazione** in cui gli oggetti contenuti (componenti/la PARTE) non hanno vita propria ma esistono in quanto parte della classe contenente (composta/il TUTTO)
- **Proprietà**
 - ❑ La **classe composta è responsabile** della **creazione** e della **distruzione** degli oggetti contenuti
 - ❑ La classe composta è l'unica a poter usare le classi componenti
 - ❑ La molteplicità dal lato della classe composta **deve** essere al più uguale ad uno, mentre può essere qualsiasi per le classi componenti



Implementazione COMPOSIZIONE

- Per tradurre tale relazione si devono
 - ❑ Definire degli attributi (privati) del tipo delle classi componenti o, in alternativa, definire una opportuna struttura attraverso la quale mantenere le istanze delle classi componenti
 - La dimensione di ogni attributo dipende dalla molteplicità
 - ❑ Istanziare le classi componenti all'interno della classe composta mediante
 - *Costruttori*, metodi *add*
 - ❑ Deallocare all'interno della classe componente mediante distruttore, o un suo sostitutivo, le classi composte

Notazione COMPOSIZIONE



Composizione: implementazione in JAVA 1/2

```
package domain;
import java.util.ArrayList;

public class Albero {
    private int tipo;
    private int aspetto;
    private int altezza;
    private Tronco strutturaPortante;
    private ArrayList<Ramo> espansione;
    // presenza e significato dei metodi costruttori ed add dipendono dalle
    // specifiche
    public void addRamo() {
        espansione.add(new Ramo());
    }
    public void addTronco() {
        strutturaPortante = new Tronco();
    }
    public Albero() {
        espansione = new ArrayList<Ramo>();
        // inizializzazioni varie
    }
}
```



Composizione: implementazione in JAVA 2/2

```
package domain;
import java.util.ArrayList;
import java.util.Collection;
public class Ramo {
    private int lunghezza, tipo;
    private Collection<Ramo> ramiSecondari; // variato al plurale il nome del ruolo
    private Collection<Foglia> organo;
    //altri attributi
    public void addFoglia() {
        organo.add(new Foglia());
    }
    public Ramo() {
        ramiSecondari = new ArrayList<Ramo>();
    }
    public Ramo(int lunghezza) {
        setLunghezza(int lunghezza);
    }
    public void addRamoSecondario(int lunghezza){
        ramiSecondari.add(new Ramo(lunghezza));
    }

    public int getLunghezza(){ return lunghezza; };

    public void setLunghezza(int lunghezza){
        this.lunghezza=lunghezza;
    };
    //altri metodi
}
```

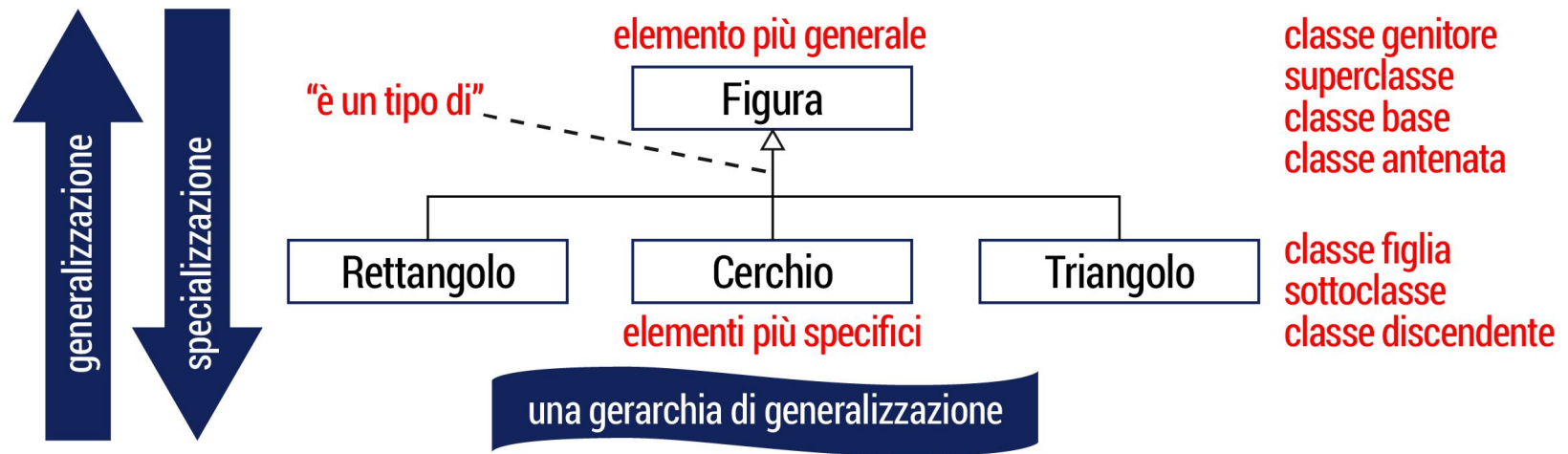


Relazione di GENERALIZZAZIONE

- La generalizzazione è **una relazione tra un elemento generico e uno specializzato**. L'elemento più specializzato è completamente consistente con quello più generico, ma contiene più informazioni
- I due elementi obbediscono al **principio di sostituibilità**
 - ❑ La generalizzazione implica il massimo grado di interdipendenza tra due elementi
- Questo tipo di relazione è molto più forte di quanto non lo sia l'associazione

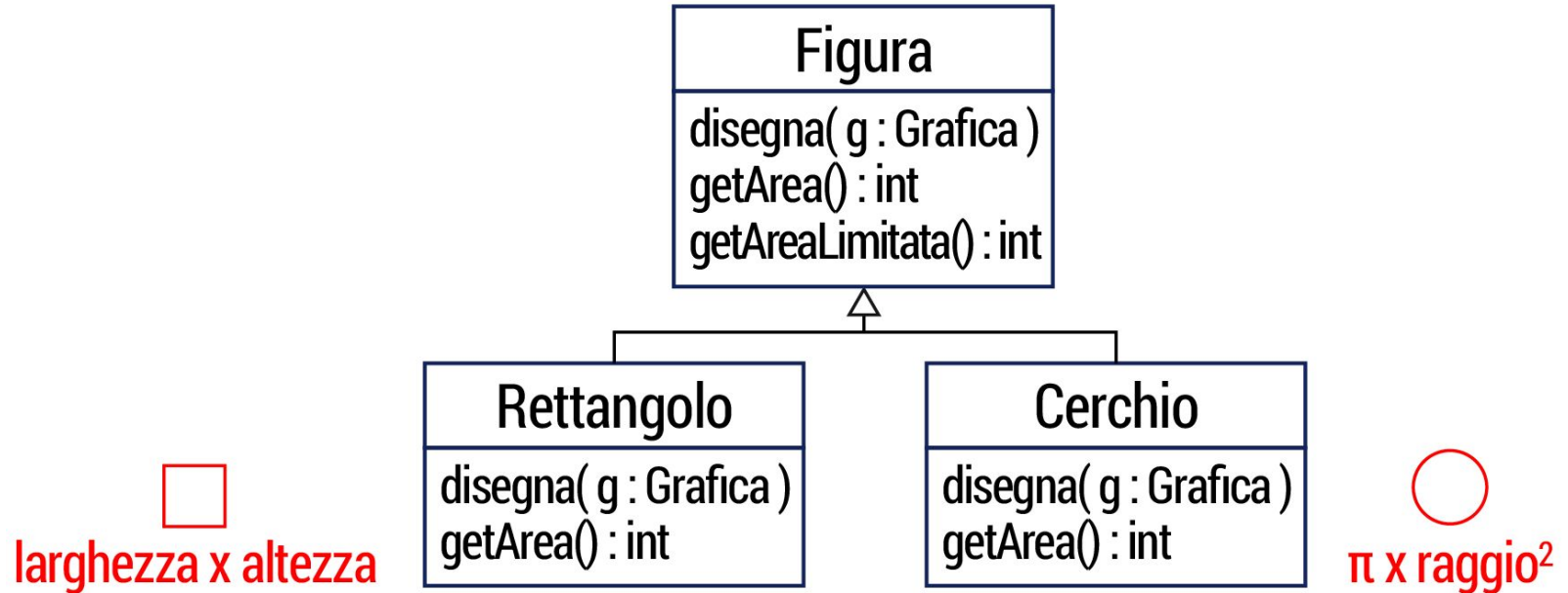


Relazione di GENERALIZZAZIONE



- Le sottoclassi ereditano:
 - **Attributi**
 - **Operazioni**
 - **Relazioni**
 - **Vincoli**
- Le sottoclassi possono ridefinire le operazioni

Relazione di GENERALIZZAZIONE

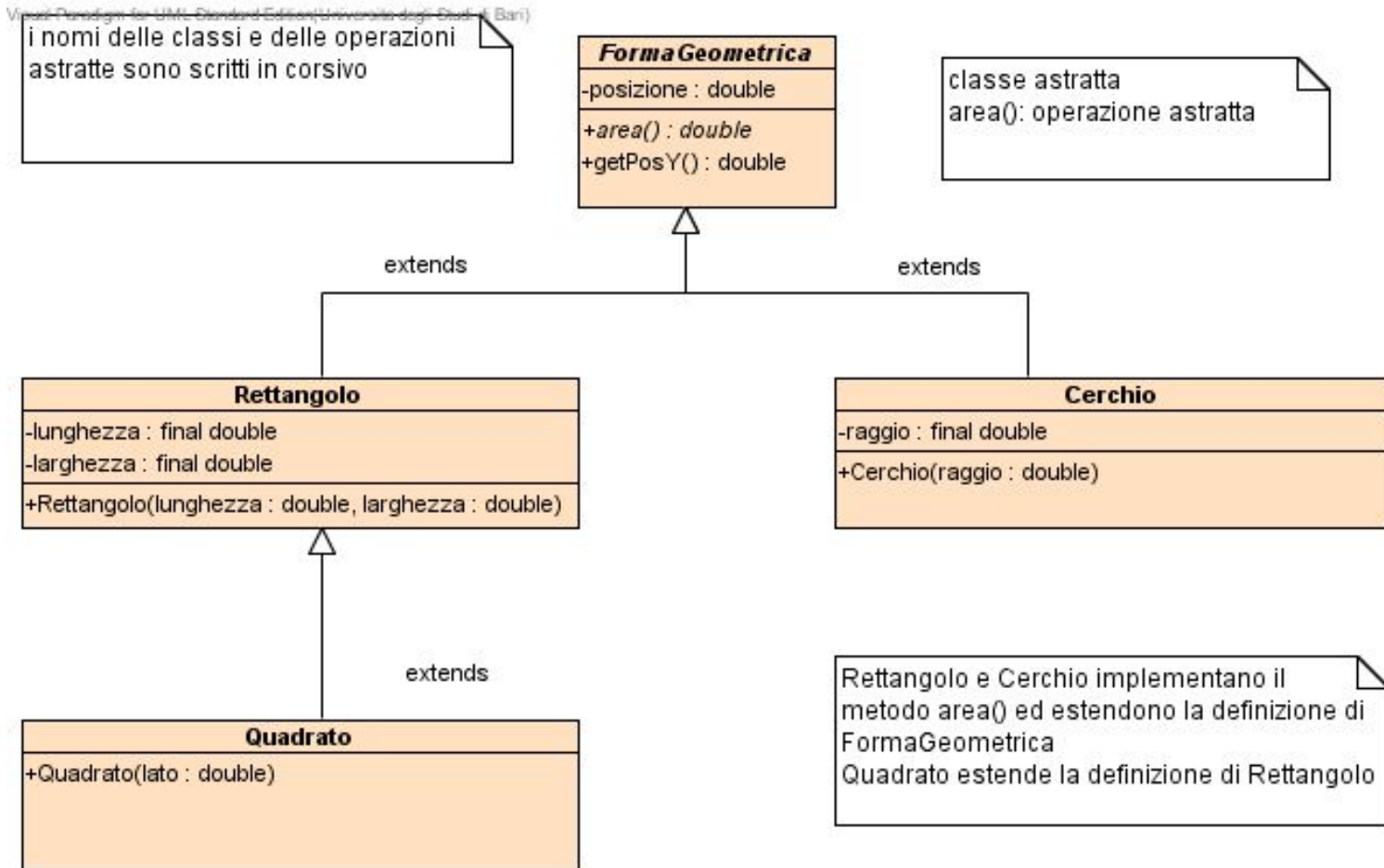


- **Rettangolo::disegna(g:Grafica)**
 - Disegna un rettangolo
- **Rettangolo::getArea():int**
 - Calcola e restituisce l'area del rettangolo
- **Cerchio::disegna(g:Grafica)**
 - Disegna un cerchio
- **Cerchio::getArea():int**
 - Calcola e restituisce l'area del cerchio

Classe Astratta

- Rappresenta una classe base (o superclasse) in una realizzazione di generalizzazione contenente uno o più operazioni astratte che **NON può essere istanziata**
- ❑ in JAVA il modificatore *abstract* è usato sia per dichiarare un classe astratta sia per dichiarare un metodo astratto

Relazione di generalizzazione: classe astratta



Relazione di DIPENDENZA

- **Definizione**

- ☐ Indica una connessione “semantica” tra due classi in cui una delle due è dipendente dall’altra

- **Osservazioni**

- ☐ Una modifica nell’elemento **indipendente** ha effetti su quello dipendente
 - ☐ Ad esempio: un cambiamento al fornitore può influenzare o fornire delle informazioni necessarie al cliente (il cliente dipende in qualche modo dal fornitore)
- ☐ La dipendenza può essere di varia natura, esistono infatti diverse relazioni di dipendenza
- ☐ Le dipendenze possono esistere tra classi, oggetti e classi, e tra *package* e *package*



Dipendenze di Uso

- «usa»
 - ❑ Il *cliente* “usa” il *fornitore*



- ❑ Un'operazione della classe A ha bisogno di un parametro della classe B
- ❑ Un'operazione della classe A restituisce un valore della classe B
- ❑ Un'operazione della classe A utilizza un oggetto della classe B da qualche parte nella sua implementazione, ma non come un proprio attributo

Riferimenti

- Ian Sommerville “Ingegneria del Software”, 10a ed. Pearson, 2017
 - Capitolo 7: Progettazione e implementazione
 - 7.1 Progettazione orientata agli oggetti tramite UML
 - 7.1.3 Identificazione delle classi di oggetti
 - 7.1.5 Specifiche delle Interfacce

Ulteriori testi consigliati

- Martin Fowler, “UML DISTILLED, 4a edizione” Pearson addison Wesley, 2010
 - Capitolo 3: Diagramma delle classi: concetti fondamentali
 - Capitolo 5: Diagramma delle classi: concetti avanzati

